

Spring Boot Interceptors — Quick Revision

1. Problem Interceptors Solve

Har controller method me common code (logging, timer, auth) baar baar likhna padta hai. Interceptor ek central jagah deta hai ye logic likhne ke liye, taki code duplicate na ho.

2. Where Interceptor Fits in Spring MVC

Request (`DispatcherServlet`) → (`HandlerMapping`) handler dhoondta hai → Interceptors chalte hain → (`HandlerAdapter`) controller ko call karta hai → Response. Interceptor "HandlerInterceptor" isliye kehlata hai kyunki handler sirf controller method nahi, static resource bhi ho sakta hai.

3. Three Lifecycle Methods

(`preHandle()`) → controller se pehle, (`postHandle()`) → controller ke baad (success par), (`afterCompletion()`) → poora request cycle khatam hone ke baad. Flow: `preHandle` → Controller → `postHandle` → `afterCompletion`.

4. Creating a Custom Interceptor

(`HandlerInterceptor`) interface implement karke (`@Component`) banate hain. Sab methods ka default implementation hota hai, jo chahiye sirf wahi override karo.

5. Method Parameters

(`HttpServletRequest`) = incoming request data (URL, headers, params).
(`HttpServletResponse`) = outgoing response (status, headers, body). (`Object handler`) = jo controller method select hua, usko (`HandlerMethod`) me cast karke controller/method ka naam nikal sakte ho (blind cast mat karo, (`instanceof`) use karo).

6. `preHandle()`

Controller call hone se pehle chalta hai. (`true`) return karo to aage badhega, (`false`) return karo to request wahi ruk jayegi aur controller call hi nahi hoga (isliye false karte time response khud likhna zaroori hai). Use: logging, auth check, timer start.

7. `postHandle()`

Controller successfully complete hone ke baad, view render hone se pehle chalta hai. Traditional MVC me (`ModelAndView`) me extra data add kar sakte ho. REST controllers me (`modelAndView == null`) hota hai kyunki response already likha ja chuka hota hai — isliye REST body `postHandle` me modify mat karo, uske liye (`ResponseBodyAdvice`) use karo.

8. afterCompletion()

Poore MVC cycle ke baad chalta hai, `finally` block jaisa. Cleanup, final logging, execution time, status check ke liye best hai. Note: `ex == null` ka matlab error nahi hua ye guarantee nahi, kyunki exception pehle hi `@ExceptionHandler`/`@ControllerAdvice` se handle ho sakta hai.

9. Registering Interceptor (WebMvcConfigurer)

Sirf `@Component` bana dena kaafi nahi hai — `WebConfig` class me `WebMvcConfigurer` implement karke `addInterceptors()` method me `registry.addInterceptor(...)` se register karna padta hai.

10. Logging Request Details

Ek practical example jisme `preHandle` me request method, URI, query string, client IP aur controller/method name print kiya jata hai. Real project me `System.out.println` ki jagah SLF4J logger use karo.

11. Authentication Demo (API Key)

`preHandle()` me `X-API-KEY` header check karke agar galat/missing ho to `401 Unauthorized` response likh ke `false` return karte hain. Yeh sirf seekhne ke liye hai, production me Spring Security use karna chahiye.

12. Authorization Demo

Similarly `X-USER-ROLE` header check karke agar ADMIN nahi hai to DELETE request ko `403 Forbidden` de dete hain. **401** = authenticate hi nahi hue, **403** = authenticated ho par permission nahi hai.

13. Measuring Execution Time

Interceptor singleton hota hai, isliye start time ko instance field me store karna galat hai (concurrent requests overwrite kar denge). Sahi tarika: `request.setAttribute()` me `preHandle` me time store karo, `afterCompletion()` me nikal ke duration calculate karo. `System.nanoTime()` use karo, `currentTimeMillis()` nahi (kyunki wall clock adjust ho sakti hai).

14. URL Patterns (Include)

`addPathPatterns("/api/students/**")` se specify karte hain ki interceptor kis URL pattern par chalega. `*` = ek segment, `**` = sab nested paths.

15. URL Patterns (Exclude)

`excludePathPatterns(...)` se specific paths (jaise login/register) ko interceptor se bahar

rakh sakte ho, wo include patterns ke saath combine ho sakta hai.

16. Multiple Interceptors & Order

Agar A aur B register hain: `preHandle()` forward order me chalta hai (A→B), `postHandle()` aur `afterCompletion()` reverse order me (B→A) — bilkul nested function calls jaisa. `.order(n)` se sequence set kar sakte ho, chota number = pehle chalega.

17. Filters vs Interceptors (Flow)

Filter → Servlet layer par pehle chalta hai (DispatcherServlet ke bahar). Interceptor → Spring MVC ke andar, handler select hone ke baad. Full order: Filter(before) → Interceptor.preHandle → Controller → Interceptor.postHandle → Interceptor.afterCompletion → Filter(after).

18. Filter vs Interceptor Table

Filter request/response ko wrap kar sakta hai (body caching, compression) — Interceptor nahi kar sakta. Interceptor ko controller method pata hota hai (`HandlerMethod`), Filter ko normally nahi pata hota.

19. Kab Filter Use Kare

Jab concern poore HTTP request par ho (body wrapping, encoding, CORS, compression, Spring Security jaisi cheeze) — controller specific na ho.

20. Kab Interceptor Use Kare

Jab logic Spring MVC handler par depend kare — jaise controller method ka naam, custom annotation check, tenant context, controller-specific timing/logging.

21. Common Mistakes

- Interceptor bana diya par `addInterceptors()` me register karna bhool gaye.
- `false` return kiya par response likhna bhool gaye (client ko blank response milega).
- Request data ko instance field me store kiya (singleton issue) — request attribute use karo.
- Handler ko blindly cast kiya bina `instanceof` check kiye.
- REST response ko `postHandle()` me modify karne ki koshish ki (already written ho chuka hota hai).
- Production security khud interceptor se banai, jabki Spring Security use karna chahiye.
- `ex == null` dekh ke maan liya ki error nahi hua.

22. Final Mental Model

Filter poochta hai: poore HTTP request ke around kya karna hai (servlet tak pahuchne se pehle/baad)? **Interceptor** poochta hai: selected MVC handler ke around kya karna hai (controller se pehle/baad)? Rule: preHandle → postHandle → afterCompletion; multiple interceptors me — pehle forward order, baad me reverse order.